PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Consumo y documentación de una API con React

Presentado a: **Instructora Yoli Alexandra Muñoz**

Por Aprendiz: **Sara Isabel Campo Calapsu**

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 24 de 10 del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de contenido

Contenido

1. Instalación y Configuración 3
2. Conexión con Express 5
3. Desarrollo de los componentes 5
3.1. Componente inicio 5
3.2. Componente información 5
3.3. Componente solicitudes 6
4. Explicación de las herramientas 6
5. Pruebas y resultados 7
6. Dificultades y soluciones 12
7. Investigación sobre consumo de la API 12
8. Conclusión 14
9. Biografías 14

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Introducción

En esta actividad se construye una interfaz web con React para que la API pueda ser desarrollada previamente con Node y Express, haciendo que el fronted permita consultar información del servidor y enviar los datos de una solicitud mediante un formulario y que su comunicación se realice con peticiones HTTP usando el fetch, haciendo que las respuestas se procesen en el formato Json.

Objetivo

El objetivo de esta actividad es desarrollar y probar una interfaz que consuma los endpoints Get/inicio, información y POST/solicitudes, utilizando los componentes como el useState, eventos, async, await y el manejo de errores, también se busca comprender como intercambian i formación el fronted y el backend.

1. Instalación y Configuración

Para hacer la instalación primeramente creamos un archivo con el comando, npm créate vite@latest actividadReact -- --template react, que crea el documento donde vamos a trabajar nuestro desarrollo de React.

```
PS C:\Users\INTEL> npm create vite@latest actividadReact -- --template react

> npx
> cva actividadReact --template react
```

Luego de haber creado la carpeta nos dará unas opciones las cuales debemos elegir en mi caso elegí el Eslint y no hice instalacion inmediata de los paquetes de dependencia del proyecto si no que la hice después, porque el node es el me permite ejecutar el JavaScript fuera del navegador

```
create-vite v9.3.1

◇  Package name:
│  actividadreact
│
◇  Which linter to use?
│  ESLint
│
◇  Install with npm and start now?
│  No
│
◇  Scaffolding project in C:\Users\INTEL\actividadReact...
│
└  Done. Now run:

  cd actividadReact
  npm install
  npm run dev
```

Luego el comando npm install es el que genera toda la estructura inicial del proyecto descargando las dependencias y el npm run dev es el que inicia el servidor de desarrollo, vite muestra una dirección local.

```
PS C:\Users\INTEL\actividadReact> npm install

added 143 packages, and audited 144 packages in 2m

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Los archivos principales son el
src/main.jsx que es el punto de entrada donde monto el componente App en el elemento root, el otro componente es el de la App que es la que organiza la interfaz y muestra los tres componentes.
Después creamos los componentes Inicio, información y solitudes que son los componentes donde vamos a trabajar, el inicio es donde se va mostrar el mensaje, en la información se va presentar las propiedades

que se van a recibir y en las solicitudes se va capturar y a neviar el nombre, apellido y correo con el POST.
En el css de la App defino la apariencia visual adaptable y en el vite. Config.js configuro el proxy local hacia la API express con el puerto 3000.

2. Conexión con Express

Primero debemos iniciar el backend desde su carpeta con el comando que configure en la actividad anterior para asi poder probar el fronted, por ejemplo, npm run dev o el npm start, la API debe estar escuchando con el puerto 300, el vite. Config.js es el que reenvía las rutas inicio, información y solicitudes a el http, que es el que permite usar rutas relativas desde React y evita tener que habilitar el CORS para esta configuración local.

3. Desarrollo de los componentes

3.1. Componente inicio

El inicio.jsx es el que consulta el endpoint GET/inicio cuando la persona presiona el botón, la función consultar Información es asíncrona porque primero activa el estado de carga y limpia los mensajes anteriores con el que después va a ejecutar el fetch(‘/inicio’), que espera la respuesta, convierte el cuerpo a JSON con response.json() y comprueba response.ok si la respuesta es correcta y ahí guarda la propiedad mensaje en el useState para mostrarla en pantalla, si hay un error presenta un aviso, el bloque finally es el que desactiva el indicador de carga tanto si la petición termina bien como si falla.

3.2. Componente información

El componente información.jsx es el que realiza la petición a GET/ información al presionar su botón, guarda el objeto JSON completo en el estado datos. Object.entries() permite recorrer las propiedades que devuelve la API y mostrarlas en la interfaz, de esta forma el

componente no depende de que exista únicamente una propiedad concreta, sino que también muestra mensajes de carga y de error.

3.3. Componente solicitudes

El componente solicitudes.jsx contiene los campos nombre, apellido y correo, el formulario almacena lo que se va escribiendo, la función cambiarCampo es el que actualiza el campo correspondiente usando el atributo name del input, cuando se envía el formulario el event.preventDefault() que evita la recarga de la página y después se validan los campos junto con el formato básico del correo, si la validación local pasa, fetch() envia una petición POST a solicitudes haciendo que la opción method establezca el método HTTP y los headers incluyan el content.Type : application/json y JSON.stringify() convierta el objeto JavaScript a una cadena JSON y así la API vuelva a validar los datos en el backend y la validación fronted mejore la experiencia pero sin sustituir la validación del servidor.

4. Explicación de las herramientas

Fetch(): Es el que inicia una petición HTTP a una ruta del servidor y devulve una promesa con el objeto Response.

Async y await: El async es el que declara una función asíncrona y el await permite esperar el resultado de una instrucción antes de continuar con la siguiente.

response.json(): Es el que lee y convierte el cuerpo en una respuesta JSON en un valor de JavaScript que también devuelve una instrucción por eso se usa await.

JSON.stringify(): Este es el que convierte un objeto JavaScript en texto JSON para incluirlo en el cuerpo de la petición POST.

useState: Es el que guarda información que cambia durante el uso de la página, por ejemplo, valores del formulario, las respuestas, los errores y el estado de carga.

onClick y onSubmit: El onClick es el que se ejecuta en una función al presionar un botón y el onSubmit es el que ejecuta la función de envió del formulario.

response.ok: Este es el que permite comprobar si el estado HTTP está en un muy rango, fetch puede recibir una respuesta 400 o 500 sin lanzar automáticamente una excepción.

Try, catch, finally: El try es el que contiene la operación, el catch es el que permite tratar errores y el finally es el que ejecuta acciones finales como quitar el estado de carga.

5. Pruebas y resultados

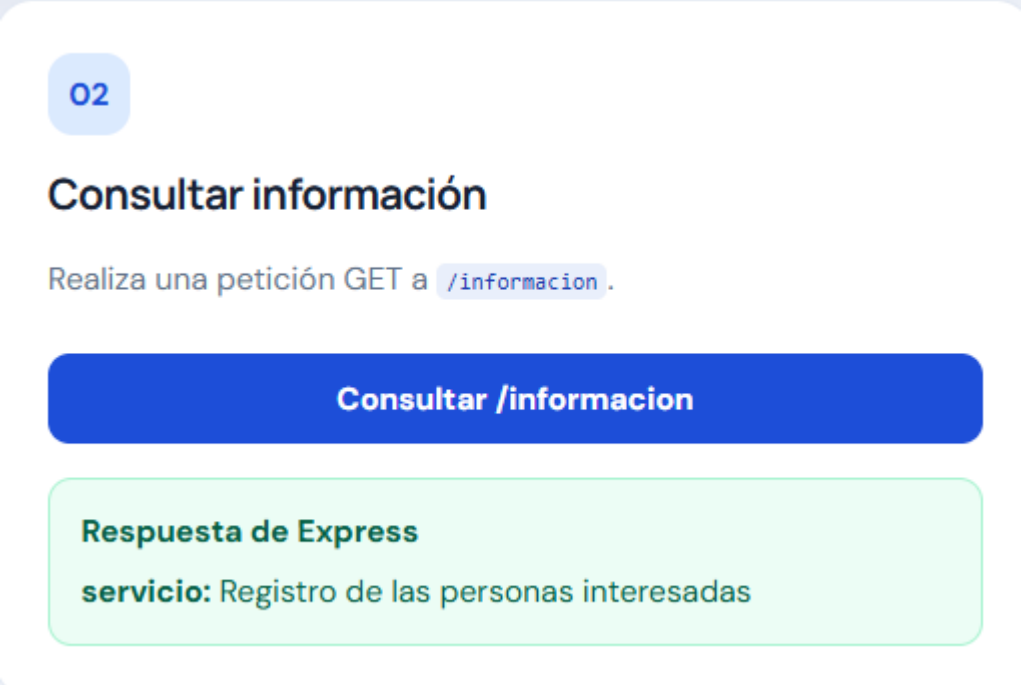
Tengo la interfaz principal de la aplicación en React ejecutándose, mostrando los componentes de consulta y el formulario de envío.



También tengo el resultado de la petición, que, al presionar el botón de consulta, mostrando el mensaje de bienvenida devuelto por la API de express.

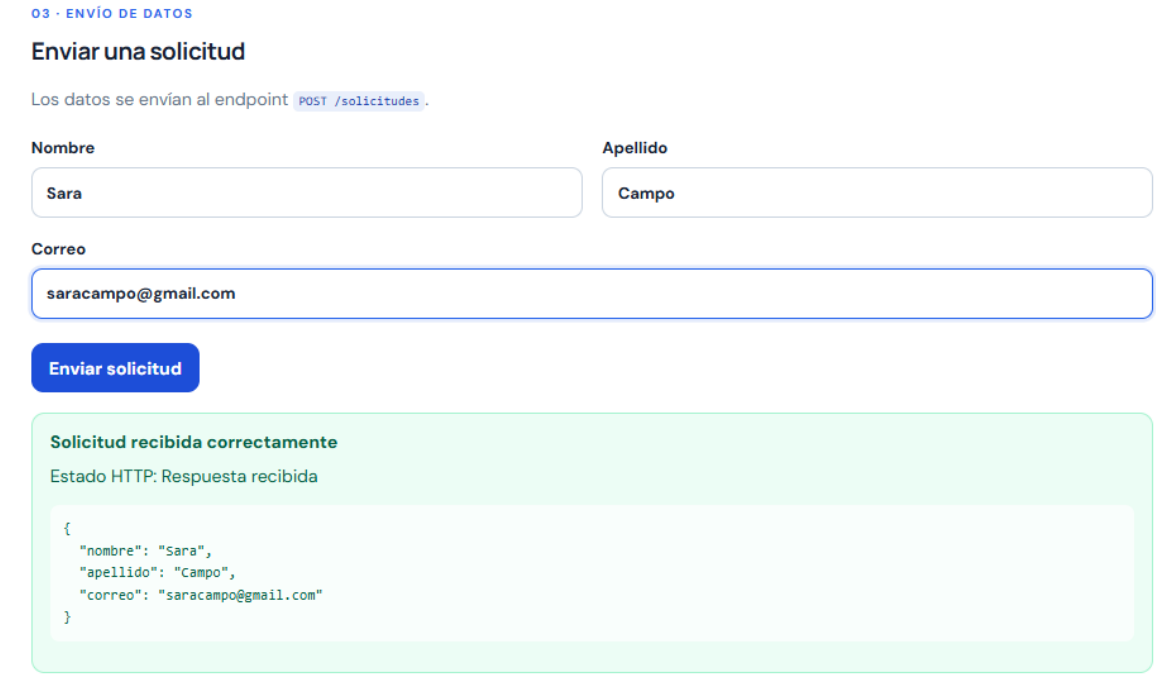


También esta la respuesta exitosa obtenida al realizar la petición, desplegando las propiedades json procesadas por el componente.

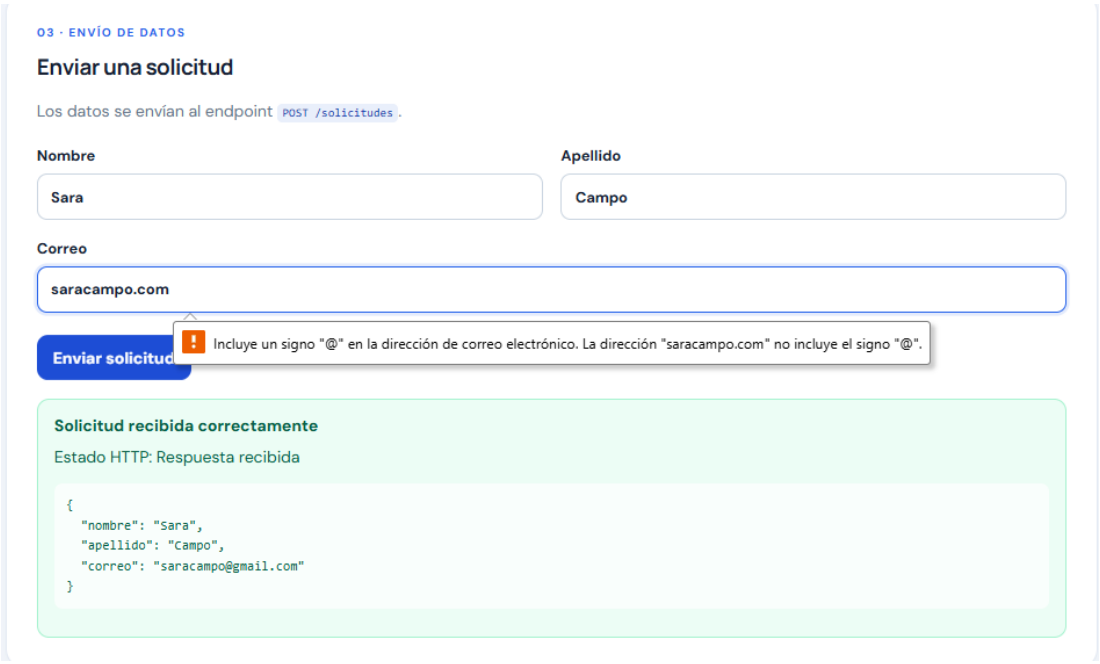


Esta el envío de solicitud mostrando el código de estado y el objeto json confirmado por el servidor.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



Tengo también la validación del formulario, mostrando el mensaje de alerta al ingresar el correo con formato invalido



Tambien esta el registro de peticiones capturando en la terminal del servidor express por medio del middleware propio

```
PS C:\Users\INTEL\OneDrive\Desktop\ExpressActividad> npm start

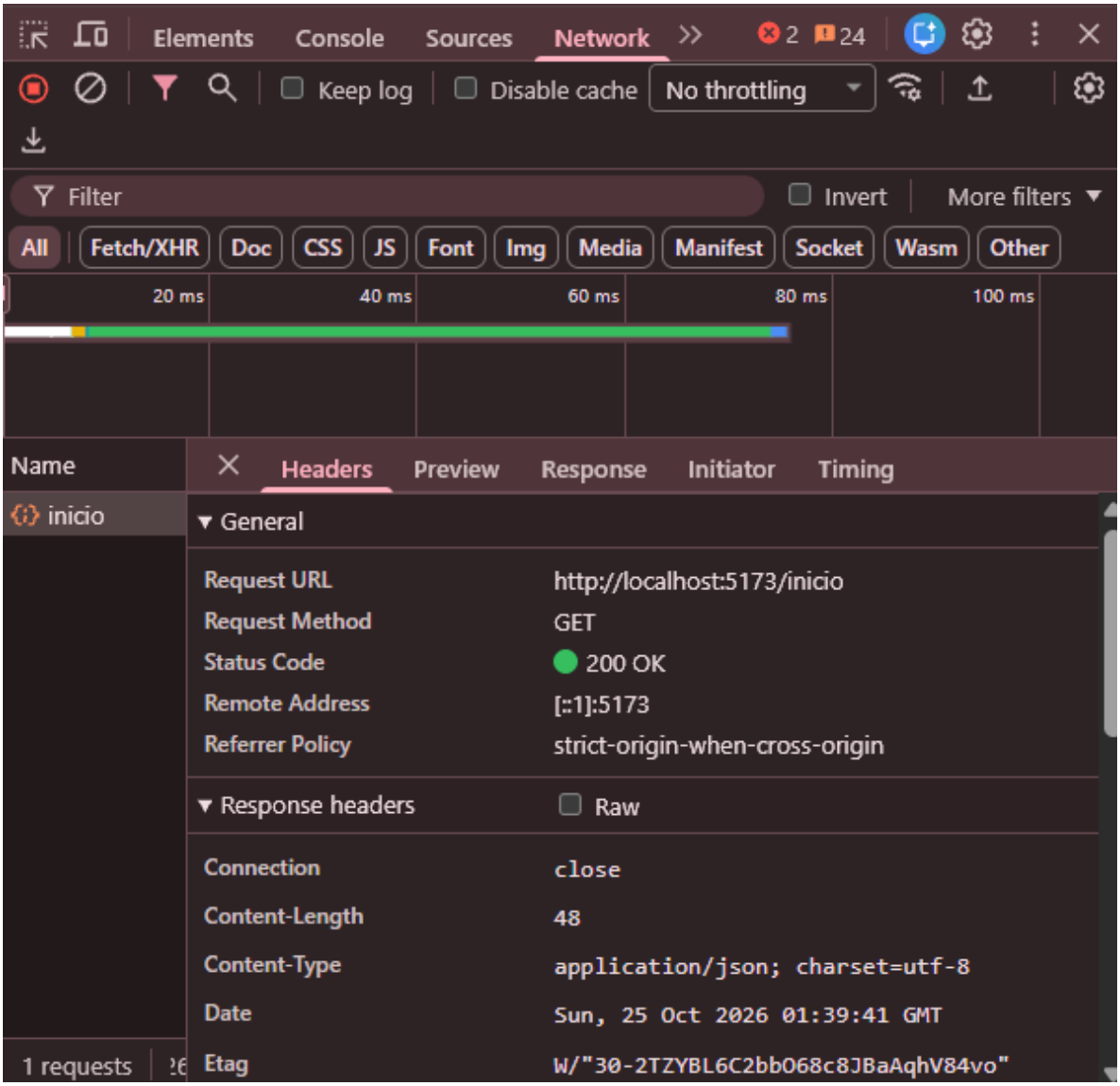
> expressactividad@1.0.0 start
> node ejercicio.js

Servidor escuchando en http://localhost:3000
GET /informacion
GET /inicio
POST /solicitudes
```

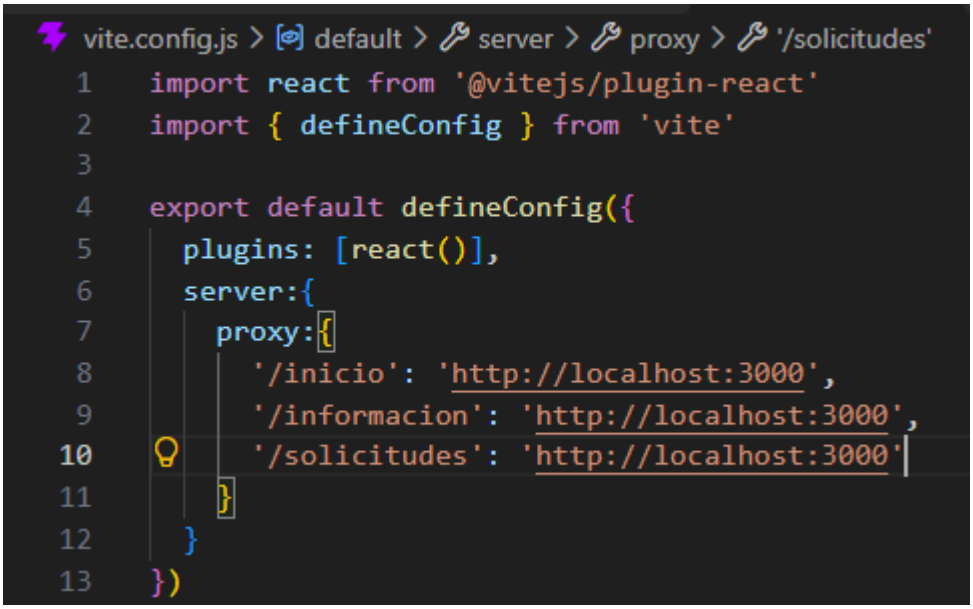
Al detener el express se muestra el mensaje de error que advierte la falta de conexión con el backend.



Al hacer petición http en las herramientas de network se valida el metodo GET, la url y el código de estado.



Por ultimo está el código correspondiente a la configuración del proxy en el vite y la implementación de peticiones con fetch() en los componentes.



6. Dificultades y soluciones

Mi primera dificultad fue al intentar consultar los endpoints desde react, porque la interfaz mostraba un mensaje de error indicando que no se podía conectar con el servidor, entonces lo que hice fue verificar que el servidor de express estuviera encendido ejecutando el comando start en la terminal, asegurándome que se ejecutara en el puerto, tal como estaba en el vite.

7. Investigación sobre consumo de la API

¿Qué es una API y para que sirve?

Es una interfaz que define cómo se comunican dos programas, en esta actividad permite que el React solicite datos o envíe información a express.

¿Qué diferencia existe entre frontend y backend?

El frontend es la parte con la que interactuamos y el backend es el que procesa las peticiones, aplica reglas y devuelve respuestas.

¿Qué significa consumir una API?

Significa que se realizan peticiones a sus endpoints y asi poder utilizar los datos o resultados que devuelve.

¿Qué función cumple fetch()?

Que permite realizar solicitudes HTTP desde JavaScript y obtener una respuesta del servidor.

¿Por qué se utilizan async y await?

Porque las peticiones tardan un tiempo variable encambio el await permite esperar el resultado de una promesa dentro de una función async.

¿Qué es una promesa?

Es un objeto que representa el resultado futuro de una operación asíncrona, que puede completarse o que también puede fallar.

¿Para qué sirve response.json()?

Sirve para leer el cuerpo de la respuesta y lo convierte desde JSON a un objeto o valor de JavaScript.

Diferencia entre GET y POST.

El GET se usa normalmente para consultar información y el POST es el encargado de enviar datos al servidor.

¿Qué son las cabeceras HTTP y Content-Type?

Son datos de la petición o respuesta, el content-Type: application/json es el que indica que el cuerpo se envía en formato JSON.

¿Por qué usar JSON.stringify()?

Porque el cuerpo de la petición debe contener texto JSON y stringify es el que convierte ese objeto de JavaScript en esa representación.

Diferencia entre error de conexión y HTTP 400.

Un error de conexión significa que no se obtuvo una respuesta del servidor y un 400 significa que sí hubo respuesta HTTP, pero el

servidor rechazó la petición por los datos enviados o que hubo una condición de solicitud incorrecta.

¿Para qué sirven useState, onClick y onSubmit?

El useState es el que conserva los datos que cambian y actualizan la interfaz, el onClick el que responde a un clic y el onSubmit el que responde al envío del formulario.

¿Qué es CORS?

Es una política de seguridad que tiene el navegador y que controla las solicitudes entre distintos orígenes, en este proyecto, el proxy de Vite es el que reenvía las rutas durante el desarrollo local.

¿Qué pasa si las propiedades no coinciden?

El backend puede no encontrar los campos esperados y rechazar la solicitud o recibir valores indefinidos, en si los nombres deben coincidir con el contrato que está en la API.

¿Por qué fetch no significa que todo salió bien?

Porque fetch puede devolver una respuesta a el HTTP 400 o 500, aunque se debe comprobar el response.ok o response.status y presentar el resultado correcto.

8. Conclusión

En esta actividad se hace mucha practica entre la comunicación de la interfaz react y de la API desarrollada con express, se aprende a consultar endpoints con el fetch() a procesar respuestas json, a poder administrar datos con el useState y a enviar información mediante el POST, también a reconocer la importancia de comprobar el estado http y mostrar errores claros, ya que una petición puede fallar por que hay datos inválidos.

9. Biografías

https://react.dev/learn/adding-interactivity

https://www.ibm.com/mx-es/think/topics/api

https://developer.mozilla.org/enUS/docs/Web/API/Fetch_API/Using_Fetch